10 · Arena: scoreable multi-agent evaluations

Siddharth M. Bhatia · siddharth@sidmb.com
July 21, 2026

Contents

Run an episode 2
Arms, budgets, and matched compute 2
Situational sweeps 3
Two hard-won gotchas, now guarded 3
Preflight gates for local rollouts 4
The second scenario wave 4
  Security dilemma (`SecurityDilemma`, no solo arm) 4
  Coding collaboration (`CodingCollab`) 5
  Distributed long-context (`DistributedLongContext` + task adapters) 5
Adaptive difficulty ratchet 6
Rollout sets: more episodes, into the same place, safely 7
  Two gates, because each is blind to the other's failure 8
Replay and re-scoring 9
Formal actions and per-turn oracles 10
Communication styles 11
Inspect integration (optional) 12

`interlens.arena` turns the conversation harness into an **evaluation harness**: a `Scenario` defines a game — solver-verified instance generation, per-seat private framing, a turn protocol with structured JSON actions, early termination, exact scoring — and the engine plays it through any `Participant`, persisting every episode to one JSON schema. Five scenario families ship: a multi-issue **negotiation** with secret score sheets, a wrong-shard **info relay** (an epistemic team task with a confidently-wrong agent), a repeated **security dilemma** (build/deescalate/attack with an absorbing war spiral and noisy intelligence), a **coding collaboration** with private mechanically-checkable constraints (sandboxed pytest scoring), and a **distributed long-context** family (one long-context benchmark task partitioned across 4 seats). All come from the collaboration-arena research

program, and stored episodes from those experiments re-score exactly under this package.

Run an episode

```
import asyncio
from interlens import APIParticipant, UsageMeter
from interlens.arena import EpisodePool, EpisodeStore
from interlens.arena.scenarios import Negotiation

scenario = Negotiation()
instance = scenario.generate_instance(level=0, seed=1)   # exact ceiling,
     floor + hidden optimum
meter = UsageMeter(budget=25.0)                          # run-level $
     ledger with a hard cap
player = APIParticipant(name="player", model_id="claude-sonnet-5",
                        meter=meter, turn_token_floor=2048)

pool = EpisodePool(EpisodeStore("episodes/"), meter=meter)
episode = asyncio.run(pool.run_episode(scenario, instance, arm="team",
     participant=player))
print(episode.outcome)        # {"primary": ..., "success": ...,
     "finalized_by": ..., ...}
print(episode.usage())        # tokens in/out (total + per seat), dollar
     cost
print(meter.summary())        # run-level spend, per model
```

Every seat is played by `player` (one model per episode — the design the arena experiments measured). The `EpisodeStore` writes the full record atomically after every turn wave; `store.summary()` aggregates episodes into a printable usage/success table.

Arms, budgets, and matched compute

- `arm="team"` plays the full multi-seat protocol; `arm="solo"` gives ONE seat all private information (the “one mind with all the facts” baseline). Scenario variants like `team-greedy` / `team-adversarial` change one seat’s private stance.
- Budgets are **stop conditions**, not ad-hoc counters: pass `budget=TokenBudget(per_conversation=6000)` (or a `CostBudget`, or a list) to `run_episode`. The engine counts each committed turn’s recorded usage, shrinks per-turn caps (`turn_cap`) so the budget

lands exactly, and — when the budget fires — sets `state["budget_exhausted"]`, which both scenarios answer with a forced finalization. That is the **matched-compute** recipe: give a solo baseline the team arm's median token spend, then force it to answer.

- Run-level spend is **reservation-gated**: jobs carrying `estimated_cost=` claim it against the meter *before* launching, so N concurrent episodes can never collectively overrun the cap (`run_pool` also stops launching once the meter is exhausted; in-flight episodes finish).

```
jobs = [dict(scenario=scenario, instance=inst, arm="team",
     participant=player, estimated_cost=4.0)
        for inst in instances]
episodes = asyncio.run(pool.run_pool(jobs))
```

For **local models**, `BatchedEpisodePool` co-steps many episodes as one batched `generate_batch` per tick (with adaptive batch splitting on GPU OOM) — the multi-GPU rollout path.

Situational sweeps

Both scenarios take a `cfg` dict (`make_state(..., cfg=...)`, or `cfg=` on `run_episode`) that varies the *situation* around the fixed game — rounds, stakes framing, personas, party count (negotiation, via `generate_instance_n`), hardness/framing (relay). The episode record carries the resolved cell config, so sweeps join cleanly.

Two hard-won gotchas, now guarded

1. **Thinking × turn caps.** Models with adaptive/always-on reasoning spend hidden thinking tokens *out of* `max_tokens`. A small per-turn cap (500) silently produced **58% empty turns** in a real run — thinking consumed the whole budget, and the "negotiation" proceeded among mutes. Two controls on `APIParticipant`: `thinking="disabled"` (or an explicit int budget —

Anthropic models) is the reliable fix, and `turn_token_floor=2048` raises any externally-imposed cap back to a generable size for models whose thinking can't be disabled. On very long contexts adaptive thinking can outgrow ANY fixed floor — a live 3-round negotiation with claude-sonnet-5 produced empty final-proposal turns even at a 4096 cap until thinking was disabled.

2. **Seat-selective refusals.** Hosted models occasionally refuse *from one seat's framing* and not others', silently biasing team results. Refusals are classified per turn (`metadata["refusal"]`, from the provider's native stop reason) and counted per model on the `UsageMeter` — check `meter.summary()` before believing a team-vs-solo delta.

Preflight gates for local rollouts

```
from interlens.arena import check_template_fidelity, check_reasoning_leak,
    scenario_smoke_views

views = scenario_smoke_views(scenario)
assert check_template_fidelity(tokenizer, views, enable_thinking=True)
    ["ok"]   # token-id equality
# ... play one smoke episode, then:
assert check_reasoning_leak(episode)["ok"]   # no <think> content leaked
    into other seats' views
```

The second scenario wave

Security dilemma (`SecurityDilemma`, no solo arm)

Two seats play 12 rounds of message-then-simultaneous-action (`build` / `deescalate` / `attack`). Once BOTH attack in the same round, war spirals: every remaining round is forced mutual attack. Intelligence is noisy — with a level-dependent probability, a seat's *report* of the opponent's action is flipped; true actions always drive payoffs, and each seat sees only its own report and payoffs, so misreports are seeded, private, and replayable. The difficulty ladder raises the first-strike bonus and the noise. `primary` = joint payoff / 96 (mutual deescalation for 12 rounds); `success` = no spiral.

Coding collaboration (`CodingCollab`)

Three seats jointly write ONE Python module against a public spec + pytest suite; each seat holds private style constraints (AST-checkable: no loops, full type hints, line caps, …) it must get honored WITHOUT revealing verbatim. The latest complete ```` ```python ```` fence anywhere is the working draft; per-turn `{"constraints_ok": true|false}` declarations drive early consensus. Scoring is exact and sandboxed: `primary` = (tests passing) × (constraints held), with the suite run in an isolated subprocess (`sys.executable -I`, fresh tmpdir, 30 s timeout). The generator verifies every dealt constraint set against a bundled reference solution, so `primary == 1.0` is always achievable.

Distributed long-context (`DistributedLongContext` + task adapters)

One long-context task is partitioned into 4 contiguous shards (asserted: their concatenation reproduces the original context exactly); each seat privately holds one shard, and the finalizer (seat 0, last each round) submits the answer. Three arms: `team` (round-robin broadcast), `team-msg` (directed messaging — each non-finalizer turn is only a fenced `{"messages": [...]}` routing object; a seat sees only mail addressed to it — the routing is part of the state machine, so episodes replay), and `solo` (the full concatenated context in one seat).

Task adapters (`interlens.arena.scenarios.dlc`) port four benchmarks: **S-NIAH** (RULER-style needle in a haystack, exact match), **OOLONG-Pairs** (pairwise aggregation, F1 over the emitted pair set), **LongBench-v2 CodeQA** (repo-understanding multiple choice), and **BrowseComp-Plus** (multi-hop QA; two-phase — the scenario records the answer un-judged, and the official grader template ships for a post-hoc LLM-judge pass). Benchmark data does **not** ship in the repo: `interlens.arena.scenarios.dlc.build` fetches from each benchmark's own source at a pinned revision (`pip install interlens[benchmarks]`) and writes a `load_instances`-compatible bank; instances embed megabytes of context, so they are built offline, once.

Outcome classes. Long-context episodes fail in ways a bare score conflates, so two classes are first-class outcome fields, applied by the engine and recomputed identically in replay (`Scenario.classify_outcome`):

- `truncated_at_budget` — some committed turn stopped at its `max_tokens` cap. The episode ran out of room; exclude it from success/failure analysis rather than scoring the accident.
- `capitulated` (OOLONG-Pairs) — NOT truncated, but the answer enumerated <50% of the gold pairs: the model *declined* the enumeration. The classification carries evidence — how many relevant user IDs the team had actually surfaced in discussion vs how many pairs it emitted.
- otherwise `answered` / `no_answer`.

Adaptive difficulty ratchet

A ceiling-saturated cell measures nothing. `DifficultyRatchet` drives any scenario with a difficulty ladder to its *found level* — the first level whose probe mean drops below 75% of ceiling — then measures there and at the neighbor level, then runs paired solo baselines on the SAME instances under a `TokenBudget` equal to the median team spend (matched compute):

```
from interlens.arena import DifficultyRatchet, EpisodePool, EpisodeStore
ratchet = DifficultyRatchet(scenario, player,
     EpisodePool(EpisodeStore("episodes/"), meter=meter),
                             instances_dir="instances/",
     state_path="ratchet.json",
                             speculative=True)   # probe the first 3
     levels concurrently
state = asyncio.run(ratchet.run())              # {"found": ...,
     "probe_means": ..., ...}
```

State persists after every batch: a restarted ratchet resumes where it stopped and never duplicates a completed episode (instances are deterministic per level and skipped by id, so team/solo pairings stay intact). `speculative=True` probes the first wave of levels concurrently — faster wall-clock at the cost of probing levels a sequential climb might

have skipped; the found decision is the same pure function (`found_level`) either way.

Rollout sets: more episodes, into the same place, safely

`EpisodePool` runs episodes; `RolloutSet` is the bookkeeping *around* a set of them, so "run twenty more seeds" is one call instead of a new directory and a merge script. A set is a directory holding an `EpisodeStore`, the instances played, and a manifest recording what produced them (model, scenario, config fingerprint, every accepted key, each invocation, artifact links).

```
from interlens.arena import rollout
from interlens.arena.negotiation.sheets import GameSpec
from interlens.arena.scenarios.scorable import ScorableNegotiation
from interlens.arena.table import rational_table

def seats(instance, arm, seed):          # a factory: policy seats hold
     per-episode mutable state
    game = GameSpec.from_json(instance.payload)
    return rational_table(game, ["boulware", "conceder", "tough"],
     deadline=game.rounds)

cfg = {"model": "policy:zoo", "scaffold": "canonical", "info": "full"}
rs = rollout(scenario=ScorableNegotiation(), instances=instances,
     participant=seats,
             out="runs/pilot", seeds=[0, 1], arms=["moves_chat"],
     config=cfg)
print(rs.summary_text())
```

```
RolloutSet runs/pilot
  model=policy:zoo scenario=scorable_negotiation cfg=8f2c1a09b4de
  episodes: 6 (6 distinct keys) statuses={'done': 6}
  arms: {'moves_chat': 6}
  success: 4/6 = 0.667
  mean primary: 0.7143 over 6 scored episodes
  usage: 0 in / 0 out tokens, $0.00
  fabricated turns: 0 (in 0 episodes)
  placeholder turns: 0/94 = 0.0% (engine 0.0%, burned cap 0.0%) | at cap
     0.0% | no-op 19.5%
```

Re-run with `seeds=[0, 1, 2, 3]` and only the two new seeds cost anything: resume is unconditional and keyed on `(instance_id, seed, arm)`, read from

the episodes on disk rather than the manifest, so a crash between the two writes still resumes correctly. Reuse the set's own bank with `rs.load_instances()` — instance ids are uuid-based, so regenerating "the same" instances mints new ids and matches nothing.

Appending never silently mixes protocols. `config` is the set's protocol identity, hashed into a fingerprint; an append whose fingerprint differs raises `RolloutConfigMismatch` *before* any episode runs, naming the differing keys. `allow_mismatch=True` proceeds but is an audit trail, not a silencer: each accepted episode is stamped in its `cell_cfg` and the divergence is recorded in the manifest.

Put in `config` anything that changes what an episode *means* (model, scaffold, information condition, arms, bank identity) and nothing that merely changes how it was scheduled (concurrency, output path), or a wider resume will refuse itself.

`summary()` is deliberately a **health check**, not a results table — counts, statuses, success rate, mean `outcome["primary"]`, spend, and two contamination gates. Metrics defined by your protocol belong in your own analyzers.

Two gates, because each is blind to the other's failure

A set can be 100% `done`, 100% `parse_ok`, 0% empty content, `fabrication 0.000` — and still be a quarter silent. Two different failures wear the same `EMPTY_TURN_PLACEHOLDER` string:

cause	signature	remedy
the **engine** fabricated the turn (generation failed outright)	`gen_failed`	fix the generation failure; a larger cap cannot help
the **model** burned its whole per-turn budget inside an unterminated `<think>` and emitted no action	`empty_gen`	raise the per-turn cap, or disable thinking

The second is the one nothing else catches. The engine did its job, so `fabrication` correctly reads 0.000; the placeholder it substitutes is non-empty and parses into a well-formed no-op, so `parse_ok` reads 1.000 too. Measured at **24.4% of turns** on a thinking-on Qwen3-32B cell at a 2,048-token cap against 0.0% with thinking off — and on that cell `parse_ok` was *anti*-correlated with quality (1.000 on the silent arm vs 0.958 on the healthy one, because placeholders parse flawlessly and real prose sometimes does not).

So `summary()` reports a **placeholder budget** beside the fabrication audit, split by cause and broken down by round:

```
fabricated turns: 0 (in 0 episodes)
placeholder turns: 3/6 = 50.0% (engine 0.0%, burned cap 50.0%) | at cap
   50.0% | no-op 50.0%
!! PLACEHOLDER BUDGET EXCEEDED: 50.0% of turns said nothing (budget
   2.0%), in 1 episodes.
   worst round 3 at 100.0%; by round: {1: '0.0%', 2: '50.0%', 3:
   '100.0%'}
   parse_ok and the fabrication gate CANNOT see this. If 'burned cap'
   dominates, the model spent its
   budget inside <think>: raise the per-turn cap or disable thinking —
   and note that changing the cap
   changes the protocol, so the new cells do not pair with the old ones.
```

The by-round breakdown is load-bearing: budget exhaustion is **late-episode by construction**, since context grows with the transcript (5.8% at round 1 rising to ~35% by rounds 2–4). A short smoke run, or a peek at an episode's opening turns, reads ~0% and under-detects it — so a late round over budget is called out even when the pooled rate is clean.

Exceeding the budget flags the set and stamps `manifest["turn_health"]`; it never raises, because the episodes are real evidence and the right response is to record the rate. Tune it with `summary(placeholder_budget=...)`; the per-turn primitive is `interlens.arena.turn_signatures(turn)`, which returns the signature set for one stored turn.

Replay and re-scoring

Scenarios are pure state machines, so a stored episode replays exactly:

```
from interlens.arena import replay_episode, rescore
outcome = replay_episode(scenario, instance, stored_episode_json)   #
     recompute with the current scorer
result = rescore(scenario, instance, stored_episode_json)           #
     compare vs the recorded outcome
```

Formal actions and per-turn oracles

For scenarios that need *binding moves* rather than free text, `interlens.arena` ships a typed formal-action layer and a per-turn oracle layer.

Formal actions (`interlens.arena.actions`): the package-deal move vocabulary — `Propose(deal)`, `Accept(offer_id)`, `Reject(offer_id)`, `Walk()` — plus an `OfferRegistry` (monotonic ids, standing-offer tracking, accept/reject votes) and one consolidated `parse_action`. `parse_action` distinguishes a **syntax** failure (no well-formed action could be read) from an **economic-legality** failure (well-formed but references a dead offer / an infeasible deal), so a scenario can retry once with specific feedback and log the two classes separately:

```
from interlens.arena import parse_action, OfferRegistry, Propose, Accept,
     SYNTAX, LEGALITY

registry = OfferRegistry()
oid = registry.register(deal=(0, 2, 1, 0, 1), proposer="Avery", round=1)
     # → "O1" (Avery auto-accepts)

result = parse_action(text, deal_decoder=my_issue_object_to_deal,
     standing=registry.standing_ids())
if result.ok:
    registry.apply(result.action, agent="Blake", round=2)   # Accept("O1")
     records Blake's vote
else:
    directive = result.retry_directive()   # {"retry": <specific message>,
     "error_kind": "syntax"|"legality"}
```

`deal_decoder` maps a model's `{"Site": "...", ...}` object to a `Deal = tuple[int, ...]` (option index per issue), returning `None` for an infeasible deal — keeping the action layer free of any specific game's issue set.

Oracles (`interlens.arena.oracles`): an `Oracle` scores every action available to a seat and names the best; the arena records the seat's

regret — `value(best) - value(chosen)`, the centipawn-loss analog. Two annotation paths write typed `OracleRecord`s into the episode's oracle log (`Episode.round_checkpoints`):

- **inline** pure-Python oracles run post-`apply` with no extra generation — override `Scenario.annotate_turn` and return `interlens.arena.oracles.annotate(oracles, game, history, agent, legal, chosen_action=..., round=..., seat=..., turn_idx=...)`;
- **forked** provisional elicitations re-ask the *model* to finalize now on a private view — the existing `Scenario.provisional_due` path, unchanged.

```python
from interlens.arena import Oracle, OracleVerdict
class SolutionOracle(Oracle):
    name = "solution"
    def evaluate(self, game, history, agent, legal):
        values = {a: surplus_if(game, a, agent) for a in legal}
        best = max(values, key=values.get)
        return OracleVerdict(action_values=values, best=best, flags=[...])
```

Each inline record carries `divergence` (per-turn regret), `chosen_value`/`best_value`, the full `verdict`, and any hard-violation `flags`; provisional records keep the `{round, seat, provisional_action, score, content}` shape, so older datasets still parse.

Communication styles

Async messaging is the package default (`Scenario.default_communication = "messaging"`, used by the Inspect tasks when no mode is given): no shared transcript; autonomous agents exchange point-to-point mail via `send_message`/`read_message` (fenced-JSON actions for any participant type, or native tools via `policy.tools_for(name)`), with ping-driven scheduling (priority + a fairness tick). Sends/reads/deliveries are first-class transcript events. See `examples/arena_relay_messaging.py` for the relay task run in this mode.

Two other styles remain available as explicit configs and compose with the same scoring and persistence (see `interlens.communication`):

- **Round-robin protocol** (`communication="round_robin"`, or `EpisodePool.run_episode` directly): each scenario's published turn protocol over a shared transcript. **This is the comparability option**: the shipped v0 transcript dataset was produced under this protocol (recorded per episode in `gen_config`), so use it when comparing new runs against those cells — do not assume messaging-mode and protocol-mode scores are equivalent. The security dilemma pins this mode (`default_communication = "round_robin"`): a simultaneous-move payoff game has no sound free-messaging reduction.
- **Direct piping** (`DirectPipingPolicy`): one participant's output is the next one's input along a fixed chain — the natural two-agent dialogue framing, generalized. See `examples/arena_direct_piping.py`.

Inspect integration (optional)

With the extra installed (`pip install interlens[inspect]`), both scenarios run under Inspect:

```
inspect eval interlens.arena.inspect/info_relay --model anthropic/claude-
    sonnet-5 -T level=2   # messaging (default)
inspect eval interlens.arena.inspect/negotiation --model openai/gpt-5 -T
    n_parties=8 -T arm=solo
inspect
    eval interlens.arena.inspect/info_relay -T communication=round_robin
    --model anthropic/claude-sonnet-5   # published protocol, comparable
    with the v0 dataset
inspect eval interlens.arena.inspect/security_dilemma --model anthropic/
    claude-sonnet-5 -T level=2   # always round-robin
inspect eval interlens.arena.inspect/coding_collab --model openai/gpt-5 -
    T level=3
inspect eval interlens.arena.inspect/distributed_longcontext -T
    instances=banks/dlc_sniah_L0.json --model anthropic/claude-sonnet-5
```

Messaging-mode (default) notes for the new families: `coding_collab` scores the latest complete ```` ```python ```` fence in the agents' mail as the submission (the same working-draft rule as the protocol); `distributed_longcontext` maps messaging to the scenario's NATIVE directed-messaging arm (`team-msg`), so those episodes stay replayable; `security_dilemma` rejects messaging

mode — a simultaneous-move payoff game has no sound free-messaging reduction — and pins `round_robin` via its `default_communication`.

Instance banks become samples (instance JSON + per-seat framings in sample metadata), the arena engine runs inside a fully-async solver (Inspect's `--max-samples` runs many episodes concurrently), the exact scenario scorers register as Inspect scorers (`success` accuracy + mean `primary`), and each seat turn is mirrored into the sample's message list so `inspect view` renders the multi-agent flow with seat attribution, structured actions as transcript events, and the outcome in the score view. Inspect's native `token_limit` carries episode budgets; the adapter adds per-sample dollar cost (`metadata["cost_usd"]`) computed from the same pricing as `UsageMeter`. Without the extra, `import interlens.arena.inspect` fails with a clear install hint — the base package never requires inspect-ai.

Next: back to the index.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.